

专业 计算机科学与技术 班级 计科 20208 班 成绩

学号 20201008303 姓名 王韵菲

实验三 ARM 汇编指令实验（二）

一、 实验目的

1. 掌握 ARM 数据处理指令的用法
2. 了解 ARM 汇编指令灵活的第二操作数，编写简单的汇编程序

二、 实验环境

硬件：PC 机 Pentium100 以上。

软件：PC 机 Windows 操作系统、ARM ASD 1.2 集成开发环境、
AXD

三、 实验内容

具体内容：

1. 用 MOV 和 MVN 指令访问 ARM 通用寄存器
2. 使用 ADD、SUB、AND、ORR、CMP 和 TST 指令完成数据的加减运算及逻辑运算。
3. 用 ADS1.2 软件仿真，单步、全速运行程序，设置断点，打开寄存器窗口（ProcessorRegister）监视运算值，打开存储器观察窗口（Memory）监视 0x40003100 地址处的值。

实验步骤：

1. 启动 ADS1.2, 使用 ARM Executable Image 工程模板建立一个工程。如 SY3
2. 建立汇编源文件 Test3.s, 然后加入工程中。
3. 编译、连接工程, 选择 Project Debug, 启动 AXD 软件仿真调试。
4. 打开寄存器窗口, 监视寄存器的值。
5. 可以单步运行程序, 可以设置、取消断点, 或者全速运行, 停止运行, 调试时观察寄存器的值, 并记录在表中。

代码主要运用各个指令对数据进行计算, 下图为加有注释的代码图。

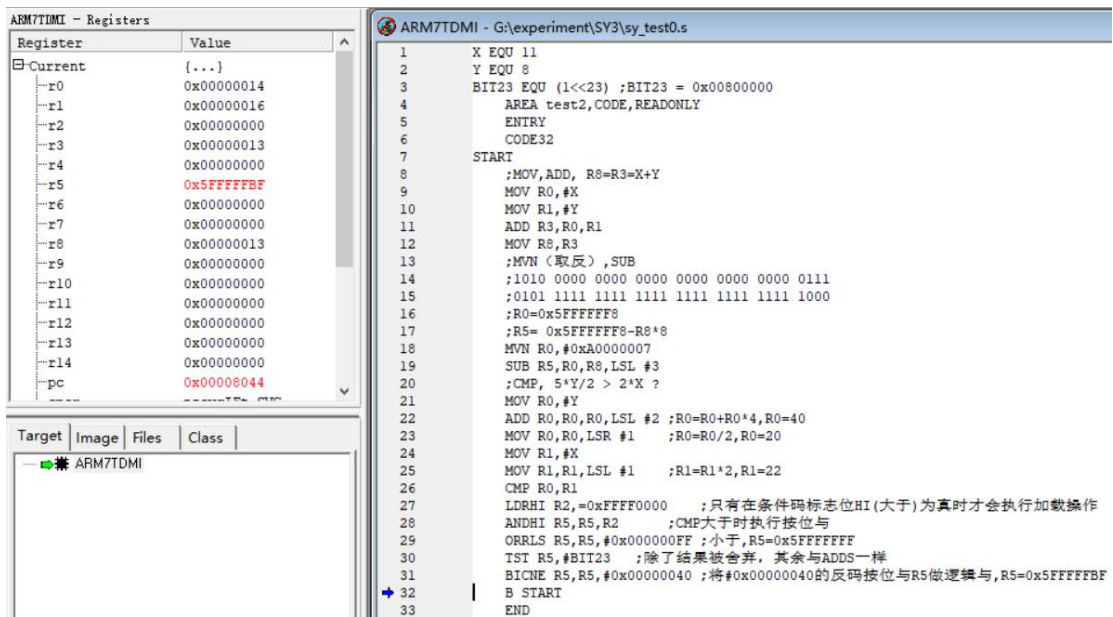


图 1 运行结果图

表 1 执行后变化情况表

序号	执行指令	执行后的变化情况						
		寄存器 (十六进制)						
		R0	R1	R2	R3	R5	R8	PC
1	MOV R0, #X	0x0000000B						0x00008004
2	MOV R0, #Y		0x00000008					0x00008008
3	ADD R3, R0, R1				0x00000013			0x0000800C

4	MOV R8, R3						0x00000013	0x00008010
5	MVN R0, #0xA0000007	0x5FFFFFF8						0x00008014
6	SUB R5, R0, R8, LSL #3					0x5FFFFFF60		0x00008018
7	MOV R0, #Y	0x00000008						0x0000801C
8	ADD R0, R0, R0, LSL #2	0x00000028						0x00008020
9	MOV R0, R0, LSR #1	0x00000014						0x00008024
10	MOV R1, #X		0x0000000B					0x00008028
11	MOV R1, R1, LSL #1		0x00000016					0x0000802C
12	CMP R0, R1							0x00008030
13	LDRHI R2, =0xFFFF0000							0x00008034
14	ANDHI R5, R5, R2							0x00008038
15	ORRLS R5, R5, #0x000000FF					0x5FFFFFFF		0x0000803C
16	TST R5, #BIT23							0x00008040
17	BICNE R5, R5, #0x00000040					0x5FFFFFFBF		0x00008044

四、 实验心得

1、写出自己的收获

在进行 ARM 汇编指令实验过程中,我通过编写代码和使用 ADS1.2 软件仿真,深入了解了 MOV、MVN、ADD、SUB、AND、ORR、CMP 和 TST 等指令的使用方法和功能。这些指令可以访问 ARM 通用寄存器,并完成数据的加减运算和逻辑运算。

通过使用 MOV 指令,我成功将常数加载到寄存器中,并且使用 MVN 指令实现了对寄存器值的取反操作。接着,我使用 ADD 指令完成了两个寄存器值的相加,并将结果保存到另一个寄存器中。使用 SUB 指令,我实现了一个复杂的减法运算,其中包括了位移操作。通过 CMP 指令,我比较了两个寄存器的值,并根据条件码标志位的结果执行了加载、按位与和按位或操作。最后,我使用 TST 指令进行了位测

试，并根据条件进行了位清除操作。

在实验过程中，我使用 **ADS1.2** 软件进行了单步和全速运行程序，并设置了断点。通过打开寄存器窗口，我可以监视运算值在不同指令执行后的变化情况。我还打开了存储器观察窗口，以监视特定地址处的值，这对于调试和验证程序的正确性非常有帮助。实验过程中出现的问题

2、实验过程中出现的问题

首先，我在使用 **MVN** 指令时，需要确保对应的操作数的位数正确，否则可能会导致错误的结果。其次，在使用 **CMP** 指令比较两个值时，需要注意条件码标志位的设置，以便正确执行后续的条件操作。此外，在进行位运算时，选择正确的指令和操作数，以确保获得期望的结果。